



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Распределенные системы управления базами данных»

Практическое занятие № 1

Студент группы *ИКБО-65-23, Олефиоров Георгий Гурамович*

(подпись)

Преподаватель *Фандеев Илья Игоревич*

(подпись)

Отчет представлен «__» _____ 202__ г.

Москва 2026 г.

СОДЕРЖИМОЕ

ВВЕДЕНИЕ	2
ПРАКТИЧЕСКАЯ ЧАСТЬ.....	5
1. Установка Docker Desktop.....	5
2. Запуск контейнера Cassandra	5
3. Подключение к cqlsh.....	6
4. Базовые команды в cqlsh	6
5. Управление контейнером	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12

ВВЕДЕНИЕ

Apache Cassandra — это высокопроизводительная распределённая NoSQL-СУБД, разработанная для обработки больших объёмов данных с обеспечением высокой доступности, отказоустойчивости и линейной масштабируемости. Она была создана в Facebook и позже передана в Apache Software Foundation. Cassandra относится к классу wide-column store (ширококолонок хранилищ) и идеально подходит для сценариев, где важны высокая скорость записи, горизонтальное масштабирование и работа без единой точки отказа.

Основные характеристики Apache Cassandra

1. Распределённость

Cassandra использует полностью распределённую архитектуру без единой точки отказа. Данные автоматически реплицируются между узлами кластера.

2. Масштабируемость

Поддерживает линейное масштабирование: добавление новых узлов пропорционально увеличивает как производительность, так и объём хранимых данных.

3. Высокая доступность и отказоустойчивость

Благодаря репликации данных система продолжает работать даже при выходе из строя отдельных узлов. Подходит для критически важных приложений, где недопустимы простои.

4. Модель данных

Cassandra использует модель column-family (семейство столбцов), где данные хранятся в виде строк с гибкой схемой. Поддерживает JSON-подобные структуры.

5. Язык запросов

CQL (Cassandra Query Language) — язык, очень похожий на SQL, но с ограничениями, обусловленными распределённой природой системы (например, отсутствие поддержки JOIN).

6. **Настраиваемая консистентность**

Поддерживает уровни консистентности: ONE, QUORUM, ALL и другие. Это позволяет балансировать между доступностью и точностью данных (теорема CAP).

7. **Производительность** Оптимизирована прежде всего для записи: данные сначала попадают в commit log (журнал), затем в memtable (память), а потом в неизменяемые SSTable на диск.

Преимущества Cassandra

- Отсутствие единой точки отказа.
- Поддержка петабайтных объёмов данных.
- Гибкая схема (schema-less в пределах column family).
- Высокая производительность как при записи, так и при чтении (особенно при правильном проектировании).

Недостатки

- Ограниченная поддержка сложных запросов (JOIN, агрегации и т.д.).
- Требуется тщательного проектирования модели данных и настройки кластера для достижения оптимальной производительности.

Области применения

Cassandra активно используется в системах, где критичны масштабируемость и отказоустойчивость:

- Социальные сети (например, Instagram).
- IoT-платформы.
- Рекомендательные системы.
- Аналитика в реальном времени.

Крупные компании, такие как Netflix, Apple, Instagram, успешно применяют Cassandra для обработки огромных объёмов данных.

Модель данных в Cassandra

В терминологии кассандры приложение работает с пространством ключей (keyspace), что соответствует понятию схемы базы данных (database schema) в

реляционной модели. В этом пространстве ключей могут находиться несколько колоночных семейств (column family), что соответствует понятию реляционной таблицы. В свою очередь, колоночные семейства содержат колонки (column), которые объединяются при помощи ключа (row key) в записи (row). Колонка состоит из трех частей: имени (column name), метки времени (timestamp) и значения (value). Колонки в пределах записи упорядочены. В отличие от реляционной БД, никаких ограничений на то, чтобы записи (а в терминах БД это строки) содержали колонки с такими же именами, как и в других записях — нет. Также в последних версиях кассандры появилась возможность выполнять запросы определения и изменения данных (DDL, DML) при помощи языка CQL.

Особенности

Запись в кассандру работает с большей скоростью, чем чтение. Это меняет подход, который применяется при проектировании. Если рассматривать кассандру с точки зрения проектирования модели данных, то проще представить колоночное семейство не как таблицу, а как материализованное представление (materialized view) — структуру, которая представляет данные некоторого сложного запроса, но хранит их на диске.

Вместо того, чтобы пытаться как-либо скомпоновать данные при помощи запросов, лучше постараться сохранить в колоночное семейство все, что может понадобиться для этого запроса. То есть, подходить необходимо не со стороны отношений между сущностями или связями между объектами, а со стороны запросов: какие поля требуются выбрать; в каком порядке должны идти записи; какие данные, связанные с основными, должны запрашиваться совместно — всё это должно уже быть сохранено в колоночное семейство.

Количество колонок в записи ограничено теоретически 2 миллиардами.

ПРАКТИЧЕСКАЯ ЧАСТЬ

В практической части работы была выполнена установка и запуск Apache Cassandra в контейнеризированной среде с использованием Docker, а также освоены базовые операции с базой данных через оболочку cqlsh. Все действия выполнялись на одном узле (single-node кластер), как рекомендуется в методических указаниях для первоначального знакомства с СУБД.

1. Установка Docker Desktop

Для работы с Cassandra в изолированной среде был установлен Docker Desktop

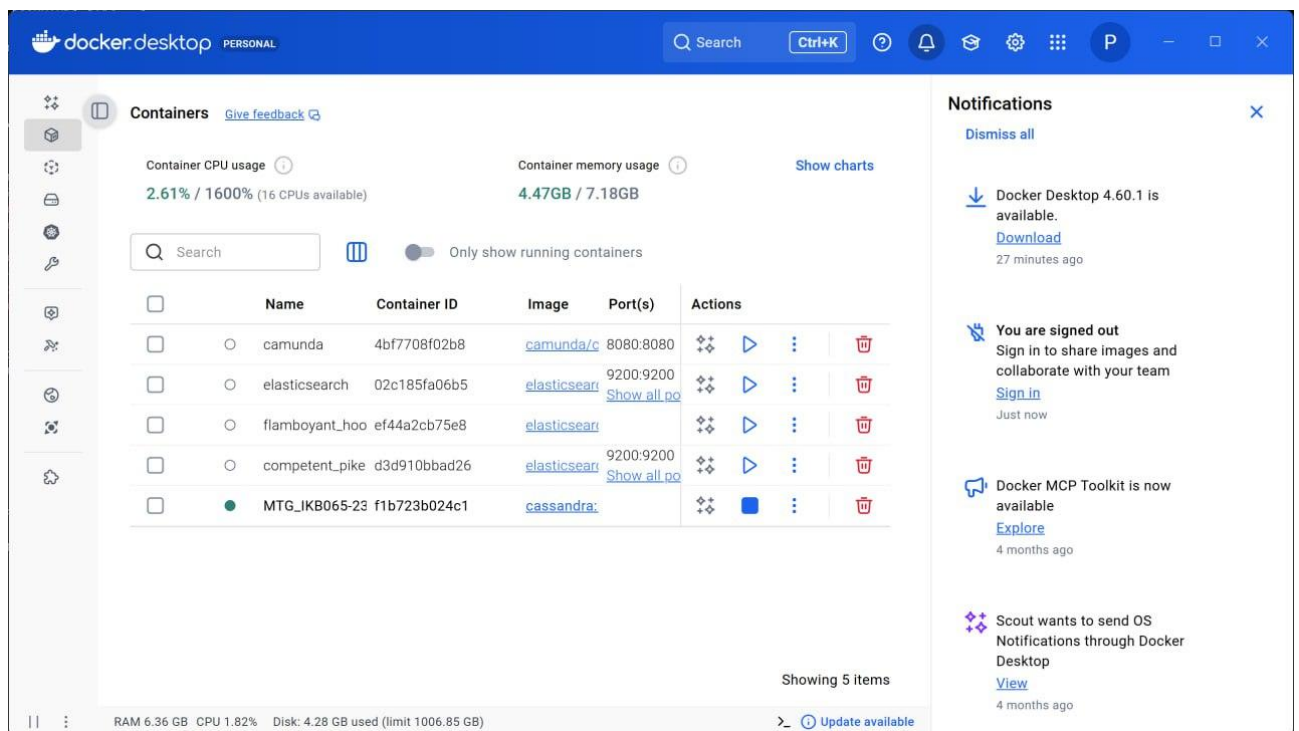


Рисунок 1 – Окно Docker Desktop после запуска

2. Запуск контейнера Cassandra

Запустил контейнер с официальным образом Cassandra:

```
PS C:\WINDOWS\system32> docker run -d --name MTG_IKB065-23 cassandra:latest
Unable to find image 'cassandra:latest' locally
latest: Pulling from library/cassandra
08cbde603e3f: Pull complete
2e09e134ccc6: Pull complete
4831516dd0cb: Pull complete
1a9c6c716127: Pull complete
d9c80eaa88c1: Pull complete
73e6d146a8f6: Pull complete
8ad2f445f4b6: Pull complete
0d69c9bfbfaf: Pull complete
Digest: sha256:1db58146a93422d329f5ff90f89ebc2b2fe28955ab3e2cd2be271167d76ba7a
Status: Downloaded newer image for cassandra:latest
f1b723b024c13cd818eca5c36be982d17a365e5991e99f95ba911ec5a2395e80
PS C:\WINDOWS\system32>
```

Рисунок 2 – Запуск контейнера Cassandra

После запуска проверил статус:

```
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
f1b723b024c1   cassandra:latest "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes  7000-7001/tcp, 7199/tcp, 9042/tcp, 9160/tcp  MTG_IKB065-23
PS C:\WINDOWS\system32>
```

Рисунок 3 – Проверка статуса контейнера

3. Подключение к cqlsh

Подключился к оболочке CQL внутри контейнера:

```
PS C:\WINDOWS\system32> docker exec -it MTG_IKB065-23 cqlsh
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh>
```

Рисунок 4 – Подключение к оболочке CQL

4. Базовые команды в cqlsh

Выполнил следующие команды последовательно (все скриншоты прилагаются):

1. Справка:

```

cqlsh> HELP

Documented shell commands:
=====
CAPTURE  CLS          COPY  DESCRIBE  EXPAND  HISTORY  PAGING  SHOW    TRACING
CLEAR    CONSISTENCY  DESC  EXIT      HELP    LOGIN    SERIAL  SOURCE  UNICODE

CQL help topics:
=====
AGGREGATES                CREATE_KEYSPACE          DROP_TRIGGER             TEXT
ALTER_KEYSPACE            CREATE_MATERIALIZED_VIEW DROP_TYPE                 TIME
ALTER_MATERIALIZED_VIEW   CREATE_ROLE              DROP_USER                TIMESTAMP
ALTER_TABLE               CREATE_TABLE             FUNCTIONS                TRUNCATE
ALTER_TYPE                CREATE_TRIGGER           GRANT                    TYPES
ALTER_USER                CREATE_TYPE              INSERT                   UPDATE
APPLY                     CREATE_USER              INSERT_JSON              USE
ASCII                     DATE                     INT                      UUID
BATCH                     DELETE                   JSON
BEGIN                     DROP_AGGREGATE           KEYWORDS
BLOB                      DROP_COLUMNFAMILY       LIST_PERMISSIONS
BOOLEAN                   DROP_FUNCTION           LIST_ROLES
COUNTER                   DROP_INDEX              LIST_USERS
CREATE_AGGREGATE          DROP_KEYSPACE           PERMISSIONS
CREATE_COLUMNFAMILY       DROP_MATERIALIZED_VIEW  REVOKE
CREATE_FUNCTION           DROP_ROLE               SELECT
CREATE_INDEX              DROP_TABLE              SELECT_JSON

cqlsh>

```

Рисунок 5 – Вывод команды HELP;

2. Информация о кластере:

```

cqlsh> DESCRIBE CLUSTER

Cluster: Test Cluster
Partitioner: Murmur3Partitioner
Snitch: DynamicEndpointSnitch

cqlsh>

```

Рисунок 6 – Вывод команды DESCRIBE CLUSTER;

3. Список keyspaces:

```

cqlsh> DESCRIBE KEYSPACES

system      system_distributed  system_traces  system_virtual_schema
system_auth  system_schema       system_views

cqlsh>

```

```

cqlsh> CREATE KEYSPACE my_keyspace WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

```

Рисунок 7 – Вывод команды DESCRIBE KEYSPACES;

4. Создание собственного keyspace:

```
cqlsh> CREATE KEYSPACE my_keyspace WITH replication = {'class':'SimpleStrategy', 'replication_factor':1};
```

Рисунок 8 – Вывод команды CREATE KEYSPACE my_keyspace WITH replication = {'class':'SimpleStrategy', 'replication_factor':1};

5. Переключение на keyspace:

```
cqlsh> USE my_keyspace;
```

Рисунок 9 – Переключение на keyspace

6. Создание таблицы:

```
cqlsh:my_keyspace> CREATE TABLE user (first_name text, last_name text, PRIMARY KEY (first_name));
```

Рисунок 10 – Создание таблицы

7. Запись данных:

```
cqlsh:my_keyspace> INSERT INTO user (first_name, last_name) VALUES ('MTG', 'RKO');
```

Рисунок 11 – Запись данных в таблицу

8. Чтение данных:

```
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='MTG';
```

first_name	last_name
MTG	RKO

(1 rows)

Рисунок 12 – Чтение данных

9. Удаление столбца:

```
cqlsh:my_keyspace> DELETE last_name FROM user WHERE first_name='MTG';  
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='MTG';
```

first_name	last_name
MTG	null

(1 rows)
cqlsh:my_keyspace> _

Рисунок 13 – Удаление столбца

10. Удаление строки:

```
cqlsh:my_keyspace> DELETE FROM user WHERE first_name='MTG';
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='MTG';

first_name | last_name
-----+-----
(0 rows)
cqlsh:my_keyspace>
```

Рисунок 14 – Удаление строки

11. Очистка таблицы:

```
cqlsh:my_keyspace> TRUNCATE user;
```

Рисунок 15 – Очистка таблицы

12. Удаление таблицы:

```
cqlsh:my_keyspace> DROP TABLE user;
```

Рисунок 15 – Удаление таблицы

5. Управление контейнером

Остановка:

```
PS C:\WINDOWS\system32> docker stop MTG_IKB065-23
MTG_IKB065-23
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\WINDOWS\system32>
```

Рисунок 16 – Остановка контейнера

Запуск:

```
PS C:\WINDOWS\system32> docker start MTG_IKB065-23
MTG_IKB065-23
PS C:\WINDOWS\system32>
```

Рисунок 17 – Остановка контейнера

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы №1 «Знакомство с Apache Cassandra» были достигнуты все поставленные цели: получено представление об архитектуре и основных принципах работы одной из самых популярных распределённых NoSQL-систем, освоены базовые навыки установки и запуска Cassandra в контейнеризированной среде с помощью Docker, а также выполнены первые операции с данными через язык запросов CQL.

Apache Cassandra подтвердила свою репутацию как мощного инструмента для работы с большими объёмами данных в условиях высокой нагрузки и требований к отказоустойчивости. Ключевые преимущества системы — отсутствие единой точки отказа, линейная масштабируемость, очень высокая скорость записи, настраиваемая консистентность и гибкая модель данных — делают её идеальным выбором для современных распределённых приложений: социальных сетей, IoT-платформ, систем рекомендаций и аналитики в реальном времени.

В практической части работы удалось:

- успешно установить и настроить Docker Desktop на Windows с использованием WSL 2;
- запустить однодневный кластер Cassandra в Docker-контейнере;
- подключиться к оболочке cqlsh и выполнить полный цикл базовых операций: создание keyspace и таблицы, вставка данных (INSERT), чтение (SELECT), частичное и полное удаление данных (DELETE, TRUNCATE), удаление таблицы (DROP TABLE);
- на собственном опыте убедиться в простоте начального знакомства с системой при использовании контейнеризации, несмотря на некоторые типичные трудности настройки Docker на Windows (память, WSL).

Выполнение заданий позволило наглядно увидеть отличия модели данных Cassandra от классических реляционных СУБД: отсутствие жёсткой схемы, ориентация на запросы (denormalization и materialized views), приоритет скорости

записи над сложными аналитическими запросами. Эти особенности требуют иного подхода к проектированию, чем в традиционных базах данных, но именно они обеспечивают высокую производительность и масштабируемость.

Полученные знания и практические навыки создают хорошую основу для дальнейшего изучения Cassandra: настройки кластера из нескольких узлов, выбора стратегий репликации, уровней консистентности, работы с партиционированием, secondary indexes, materialized views и интеграции с приложениями.

Таким образом, практическая работа подтвердила, что Apache Cassandra — это не просто ещё одна NoSQL-база данных, а специализированное решение для задач, где критичны объём данных, скорость записи, доступность 24/7 и возможность горизонтального роста без существенных архитектурных изменений.

Работа выполнена полностью в соответствии с методическими указаниями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32 — 2017 Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила и оформления — URL: https://www.rea.ru/ru/org/managements/orgnirupr/Documents/gost_7.32-2017.pdf (дата обращения 07.11.2025)

2. Требования к оформлению отчетов по практикам, курсовых работ и ВКР бакалавров и магистров. : методические указания / Ю. В. Кириллина, А. Д. Лагунова, Е. Г. Бергер .— М. : РТУ МИРЭА , 2022 — URL: https://ibc.mirea.ru/books/download_resource/SHARE/54505/ (дата обращения 07.11.2025)